



PDF Download
3408877.3432404.pdf
10 February 2026
Total Citations: 4
Total Downloads: 251

 Latest updates: <https://dl.acm.org/doi/10.1145/3408877.3432404>

RESEARCH-ARTICLE

Peer Instruction in Software Testing and Continuous Integration

BHUVANESWARI GOPAL, University of Nebraska–Lincoln, Lincoln, NE, United States

STEVE COOPER, University of Nebraska–Lincoln, Lincoln, NE, United States

Open Access Support provided by:

University of Nebraska–Lincoln

Published: 03 March 2021

[Citation in BibTeX format](#)

SIGCSE '21: The 52nd ACM Technical Symposium on Computer Science Education

March 13 - 20, 2021
Virtual Event, USA

Conference Sponsors:
SIGCSE

Peer Instruction in Software Testing and Continuous Integration

Bhuvana Gopal

Computer Science and Engineering
University of Nebraska-Lincoln
Lincoln NE USA
bhuvana.gopal@unl.edu

Stephen Cooper

Computer Science and Engineering
University of Nebraska-Lincoln
Lincoln NE USA
stephen.cooper@unl.edu

ABSTRACT

Peer Instruction (PI) is an active learning pedagogy in which students actively participate in the classroom. There have been several research studies regarding the value of PI in computer science. The present work adds to this body of knowledge by examining outcomes from an undergraduate software engineering course with specific focus on the effects of PI on student learning in the topics of unit testing, integration testing and continuous integration. We find encouraging increases in levels of success as measured through a cognitive pre- and post-course survey for those topics. This work also documents and hypothesizes reasons for the cognitive gains from PI, as well as student attitudes towards PI.

CCS CONCEPTS

• Software and its engineering → Software creation and management • Software and its engineering • Social and Professional Topics → Professional Topics → Software engineering education

KEYWORDS

Software engineering, Peer Instruction, Unit Testing, Integration Testing, Continuous Integration, Active Learning, DevOps, Software Testing, Pedagogy, Software Engineering Education

ACM Reference format:

Bhuvana Gopal and Stephen Cooper. 2021. Peer Instruction in Software Testing and Continuous Integration. In *Proceedings of ACM SIGCSE Conference (SIGCSE'21). March 13–20, 2021, Virtual Event, USA*. ACM, New York, NY, USA. 7 pages. <https://doi.org/10.1145/3408877.3432404>

1 Introduction

The Australian software testing firm Tricentis estimates the cumulative cost of software bugs, glitches and security failures worldwide last year at a whopping \$1.1 trillion [6]. It is notable that software testing continues to be the primary approach used in practice to ensure the development of high quality software.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.
SIGCSE '21, March 13–20, 2021, Virtual Event, USA.

© 2021 Association of Computing Machinery.

ACM ISBN 978-1-4503-8062-1/21/03...\$15.00.

DOI: <https://doi.org/10.1145/3408877.3432404>

It is also estimated that more than 60% of the development cost is spent on testing for many software systems [39].

The ACM CS 2013 report [1] describes the need for software testing to be an integral part of the undergraduate CS curriculum. This curricular guideline document recommends the incorporation of software testing in several undergraduate CS courses. Unit testing is specifically recommended as an essential topic in Software Development in Software Engineering (SE) courses [22].

This paper presents the design of and results from a pilot study utilizing an active learning pedagogy, Peer Instruction [30], in comparison with lecture-based content delivery in an SE class. The organization of this paper is: Section 2 contains an overview of related literature. Section 3 poses research questions we set out to answer. Survey design and methodology are introduced in Section 4 and results presented in Section 5. Results are analyzed in Section 6, and Section 7 concludes.

2 Related Literature

Wong [38] reports that most software testing is done by software engineers with only a basic background in testing, mostly restricted to the application of a small set of testing tools. Knowing how to apply a few testing tools cannot substitute for a strong foundation in software testing principles and methodologies. Many of the people responsible for testing software may not be adequately prepared for the task at hand. Our colleges are initially responsible for teaching our students how to test software [8]. This is a root cause of the current state of software testing, and it is here we need to begin to remedy the problem.

2.1 Teaching software testing

Drake [13] reported on the lessons learned and challenges in teaching software testing in their Introduction to Computing course, SE course, and Software Testing course. Key takeaways from this study were that requirements were the key to software development, and that testers need clear and exact requirements to write a test. They also found creating test cases to be creative work, and that this creativity was usually not as celebrated or visible as developers' creativity. Testers need to know the domain well. Finally, a continuing relationship with industry was key to being current with testing practices.

Bertolino [7] classified the challenges facing software testing research into six areas: the why, how, how much, what, where and when of software testing. They found that creating an explicit test hypothesis, measuring test effectiveness, automating compositional testing, testing based on an empirical body of evidence, model based testing and antamodel based testing,

precision and effectiveness of test oracles, domain specific testing, generation of test input, and online testing were the main challenges facing software testing research.

Surveys from studies around the globe have shown that developers understand the importance of software testing education [18]. Yet many of them still lack formal testing related training and education [10,31]. Orso and Rothermel [32] found similar challenges in software testing research, including testing modern real world systems, oracles, testing non-functional properties, domain based testing. They also include a newer challenge on how to leverage software testing techniques for cloud-based computing.

Wong [39] reported that teaching students about testing after they have already acquired a foundation in programming is ineffective. Students with well-developed testing skills become better software engineers as the act of testing also forces the integration, application, and enhancement of the other critical software development skills such as analysis, design and implementation.

Goldwasser [19] discussed experiences in which students of a programming course were asked to submit both an implementation as well as a test set. A portion of a student's grade was then devoted both to the validity of a student's program on others' test sets: as well as how that student's test set performed in uncovering flaws in others' programs. Clarke explored how to teach software testing, based on students' use of a Web-Based Repository of Software Testing Tools [11] that contains tutorials on software testing concepts and tools, with encouraging results regarding improvement in students' understanding and use of testing techniques and tools.

Aniche et al. [4] proposed a pragmatic software testing course based on key principles to be taught to computing students, including building a test mindset and interaction with practitioners. They discovered common student misconceptions and common mistakes students make when learning software testing. According to their study, understanding testing concepts was an important area that students had trouble with. They also found that a large percentage of students (72%) perceived activities that involve practitioners, such as guest lectures from industry as important.

Buffardi and Edwards [9] aimed to observe common testing behaviors and investigate how the feedback influences those behaviors, through an adaptive online feedback system. Jones [24] presented approaches to giving all students multiple, incremental exposures to software testing throughout the curriculum, including an integrated approach that combined common test experiences in core courses, an elective course in software testing, and volunteer participation in a test laboratory.

Agarwal et al. [3] reported their experiences in designing a series of adaptive modules using NetCoach, together with a preliminary assessment of the effectiveness of the first module. Edwards [16] reported an approach to fixing errors and debugging based on a *reflection in action* strategy by relying on software testing, by changing the way assignments are assessed—where students are responsible for demonstrating correctness through testing.

In teaching continuous integration and Continuous Delivery, studies by Eddy [15] and Greising [21] have explored ideas of setting up pre-configured pipelines to teach how to deploy code and integrate code branches in an automated fashion, into a main pipeline. However, the impact of different pedagogical techniques such as PI on teaching these topics is yet to be explored.

2.2 Peer Instruction (PI)

PI was originally developed by Mazur [30] in 1991. PI is an evidence-based pedagogical practice where students are asked conceptual questions during class time, and their answers are collected through student response systems known as clickers or response cards. Physics, chemistry and biology education researchers have adopted and adapted PI to research the effectiveness of PI and to better understand the intricacies of its implementation details.

PI is relatively new to computer science education, having been studied since 2000 [29, 36–38]. Studies have shown that PI is well-liked by both students and instructors [26,29,34]. Several studies [12,29] have shown that students learn from the discussion phase when isomorphic questions [36] are used. PI has been shown to improve student efficacy [25] and to reduce fail rates [34]. It has also been shown to help in identifying struggling students early [37]. PI has been shown to be valued by students [25,33,34] and instructors [35], and result in individual learning [33,34].

While most studies to date using PI in CS have been in introductory programming courses [25,29,34,35,37,40], PI is making inroads into upper division computing courses [28] such as computer architecture and the theory of computation as well as in cybersecurity [37]. In SE, PI has been studied in the context of student discussions [2] and risk management [17].

We believe that PI can be an effective pedagogy in teaching software testing and DevOps. The contribution of this pilot study is to examine the use of PI in SE by focusing on the industry-ready topics of unit testing, integration testing and continuous integration.

3 Research Questions

In this paper, we describe a study conducted with undergraduate students using PI in an SE class using clickers [7]. Our research questions for this study were:

RQ1: Does PI help students in an undergraduate SE class learn the concepts of unit testing, integration testing and continuous integration better than students in a purely lecture format class?

RQ2: Does PI increase the enjoyment of students in learning computer science concepts in an undergraduate SE class compared to students in a lecture classroom?

4 Survey Design and Methodology

We setup an experiment consisting of two groups of students at a large Midwestern R1 University. We used a quasi-experimental pretest-posttest hybrid between groups and within groups design for this study. The control and treatment groups consisted of successive cohorts of honors sophomores from a variety of majors

including CS, Computer Engineering, Math, and Business. This SE course was a mandatory component of their cohort-based curriculum. The treatment group used PI while the control group received instruction through traditional lectures. The treatment group and control group were instructed in consecutive semesters by the same instructor with the same content.

This research project was determined to be exempt by our college's Institutional Review Board. We administered a survey to students at the beginning of the course and again at the end of the course. The survey had two sections: a cognitive portion and an affective portion. The authors of this paper created our own questionnaire for the cognitive portion of the survey. There were three sub sections in the cognitive portion – unit testing, integration testing, and continuous integration. For the affective portion of the survey, we utilized a validated instrument to measure student attitudes by Hoegh and Moskal [23], which included confidence, interest, gender, usefulness and professionalism. This instrument to measure student attitudes was identical for both the control and treatment groups. Students were not compensated for their participation in the surveys.

4.1 Cognitive instrument - pre-post survey

The cognitive portion of the survey had 16 questions. The treatment group answered these questions with open-ended responses; the control group answered them with multiple-choice responses. The survey instrument was the same for both groups, consisting of identical questions in the same order.

For the open-ended responses from the treatment group, we used multiple graders to grade these questions [5]. In 1960, Cohen introduced the kappa coefficient to measure chance-corrected nominal scale agreement between two raters [14]. Open-ended answers were graded by two graders: the primary author of this paper, and a practicing industry expert in software engineering with over 15 years of experience. We established a detailed rubric for grading each question, where we awarded one point for a correct answer, and zero points for an incorrect answer. Before grading commenced, both graders agreed on acceptable variants of keywords that were required in each correct response. Neither grader had knowledge of whether it was the pre-test or post-test, or control group or treatment group that they were grading. Both graders then graded each response from each student individually. After grading was complete, scores from both graders were compared and used to compute the inter rater reliability score between the two graders using Cohen's Kappa value, K [14]. Our results on the range of K values we observed are described in Section 5.

We analyzed the rich set of open-ended responses from the treatment group based on response keywords, classified high frequency incorrect answers, and formulated distractors from them for each question. We were thus able to convert each open-ended question to a multiple-choice question with distractors based on common misconceptions we found in our data. This was done to eliminate the need for interrater reliability and for ease of grading in subsequent iterations of the course [13]. We utilized this version of the instrument with multiple-choice responses for the control group. Grading for multiple-choice responses was done

by awarding one point for a correct answer, and zero points for an incorrect answer.

We present the set of questions utilized in the cognitive instrument below:

Questions on unit testing:

Q1: How does a unit test pass or fail?

Q2: What are the three unit test phases? Briefly explain each.

Q3: What does it mean when a test passes individually and fails when run along with other tests in a test suite?

Q4: What kind of external resources (networks, databases) should a unit test be dependent on?

Q5: How does coupling affect testability?

Q6: How does the Single Responsibility Principle affect testability?

Questions on integration testing:

Q7: How is integration testing different from unit testing?

Q8: How can you test a database without actually using the database?

Q9: What is the difference between stubs, mocks and fakes in integration testing?

Questions on continuous integration:

Q10: Describe continuous integration in a nutshell.

Q11: Why do we need to integrate regularly?

Q12: What do we integrate regularly in continuous integration?

Q13: What are some typical activities you do during continuous integration in the after-success phase?

Q14: What are some typical activities you do during continuous integration in the after-failure phase?

Q15: What are some common reasons why a build might hang?

Q16: How would you solve the following build problem: "My tests broke but were working yesterday"?

In choosing questions for the cognitive portion of the survey, we incorporated questions that were reflective of what the primary author had observed in her decades of software engineering experience in the industry as being important for beginning software engineers to know in practice. We tried to ensure that the questions required higher order thinking skills [8,27] to be answered correctly.

4.2 PI Methodology in class

Our implementation of PI in the treatment group was based on and follows the model used by Porter et al. [35, 36]:

1. Before class: Students complete preparatory readings, to learn basic concepts or definitions before class, so that these do not have to be presented in class, thus creating time for student engagement. A graded short quiz was given before each class period before the instruction started.

2. During class: Students engaged with questions designed to help them confront and explore challenging concepts [28,36]. These were multiple-choice questions built into the lecture content and students gain credit for answering the questions with a clicker. The sequence of steps for a clicker question episode in class was:

- a. Initial vote: Question posed, and students answer individually (results not displayed for class). The correctness threshold of responses in this initial response to warrant a small group discussion, varied from 30 to 70%, as per Mazur's guidelines [30].

b. Discussion in small groups (2-3 students): Students discussed their analyses and shared their reasoning with each other, perhaps convincing their groupmate(s) to change their responses for the second vote.

c. Second vote: Students answer a second time, perhaps changing their answer based on group discussion.

d. Instructor led discussion: The instructor led a class-wide discussion by asking students to share explanations and discussions they had in their group and providing clarification of how the question can be analyzed. The correct answer was clearly indicated. The results of student responses were displayed at this point. We put forth significant effort in the classroom to explain to students why PI was being used and why it could be beneficial for them, following suggestions by Simon et al. [35, 36]. The PI questions we administered are published on our webpage [20].

5 Results

5.1 Cognitive

For the treatment group, the scores for each question from both graders was used to compute the inter rater reliability score between the two graders using Cohen's Kappa value, K [14]. Our results showed a K value ranging from 0.87 to 0.97, indicating a strong agreement between the two raters on the scores for the open-ended questions. For the control group, which utilized multiple choice questions, grading was done by awarding one point for correct answers, and zero points for an incorrect answer.

We found that students in the treatment group had larger increases in scores. There were 33 students in the treatment group and 34 students in the control group. Table 1 shows what percentage of students answered each question correctly for the pre and post surveys. For example, if 10 students got a Q2 right in the treatment group for the pre survey, the value would be 0.3, 10/33.

The response scores data from each question were analyzed for normality and we found the skew of each of the distributions was close to zero. However, the kurtosis was outside the acceptable range for a normally distributed curve, as was the shape of the curve itself. As a result, we ran nonparametric one-way analysis of variance tests on the four different distributions – pre-control vs pre-treatment; pre-control vs post-control; pre-treatment vs post-treatment; and post-control vs post-treatment. We ran Kruskal-Wallis tests on all the distributions to determine statistical significance since the assumption of homoscedasticity and normality were not met on any of the distributions. We report all our results at a significance level of $\alpha = 0.05$.

5.1.1 Pre-control vs pre-treatment: In comparing learning outcomes on the basis of the scores on the pre survey for the control group versus the treatment group, we found that the two groups had similar starting points in the topic of unit testing (pre-test: $\chi^2 = 0.45$, $p = 0.52$), integration testing ($\chi^2 = 0.43$, $p = 0.51$) and continuous integration ($\chi^2 = 0.11$, $p = 0.75$), the knowledge levels where the two groups started off was not statistically different. An overall analysis of the pre-survey scores for both groups yielded a similar result ($\chi^2 = .0018$, $p = 0.75$, $p = 0.97$).

Treatment (n=33)

Control (n=34)

Scores	Pre	Post	Pre	Post
Unit Testing				
Q1	0.64	0.97	0.32	0.47
Q2	0.3	0.48	0.29	0.42
Q3	0.52	0.88	0.38	0.47
Q4	0.3	1	0.32	0.38
Q5	0.21	0.97	0.29	0.32
Q6	0.18	0.97	0.15	0.32
Q1-Q6 Average	0.33	0.88	0.29	0.38
Integration Testing				
Q7	0.52	1	0.47	0.5
Q8	0.3	1	0.26	0.32
Q9	0	0.52	0	0.18
Q7-Q9 Average	0.27	0.84	0.24	0.33
Continuous Integration				
Q10	0.06	0.88	0.05	0.15
Q11	0.12	0.88	0.10	0.15
Q12	0	0.85	0	0.10
Q13	0	0.76	0.18	0.18
Q14	0.09	0.88	0.05	0.18
Q15	0.27	0.76	0.29	0.35
Q16	0.69	0.76	0.38	0.59
Q10-Q16 Average	0.17	0.82	0.15	0.24
Overall Average	0.25	0.84	0.23	0.33

Table 1: Percentage of students who answered each question correctly

5.1.2 Post control vs post treatment: Comparing learning outcomes on the basis of the post survey scores for the control group versus the treatment group, our results showed that the two groups differed significantly on how much they knew, as evidenced by their overall post survey scores: ($\chi^2 = 37.77$, $p < 0.0001$). Our results of the Kruskal-Wallis tests for each category, at an α level of 0.05 are: unit testing ($\chi^2 = 4.47$, $p = 0.0344$); integration testing ($\chi^2 = 5.01$, $p = 0.0253$); and continuous integration ($\chi^2 = 6.63$, $p = 0.01$). Students in the PI class scored significantly higher than students in the purely lecture-based class. Figure 1 shows the distribution of overall scores in the post-treatment group and post-control group.

5.1.3 Pre vs post treatment group: Based on our results, we found that students in the PI class (treatment group) scored significantly

higher on their post survey than in the pre survey, as evidenced by their scores ($\chi^2 = 45.27$, $p < 0.0001$). Our results of the Kruskal-Wallis tests for each category, at an α level of 0.05 are: unit testing ($\chi^2 = 4.39$, $p = 0.04$); integration testing ($\chi^2 = 8.19$, $p = 0.003$); and continuous integration ($\chi^2 = 9.81$, $p = 0.002$). Figure 2 shows the distribution of scores in the pre and post-test for the treatment group.

5.1.4 Pre vs post control group: We found that there were no statistically significant differences in the overall cognitive scores pre and post survey for the control group ($\chi^2 = 0.84$, $p = 0.36$). There was also no evidence of significant learning in any of the three categories as found by our analysis: Unit testing: $\chi^2 = 3.72$, $p = 0.40$, Integration testing: $\chi^2 = 0.4286$, $p = 0.51$ and Continuous integration: $\chi^2 = 2.79$, $p = 0.1$.

5.2 Affective

With the affective portion of the questionnaire [23], both groups started out with no statistical difference in their pre test scores; however, the treatment group scored higher overall in the post test ($\chi^2 = 3.94$, $p = 0.04$). In analyzing these score differences individually for each construct, there were no statistically significant differences for the confidence construct, interest construct or the professional construct; however, the gender construct ($\chi^2 = 1.15$, $p = 0.03$) and the usefulness construct ($\chi^2 = 1.16$, $p = 0.03$) showed that the treatment group had a positive shift in attitudes regarding gender in CS and the usefulness of CS after PI compared to their control group counterparts.

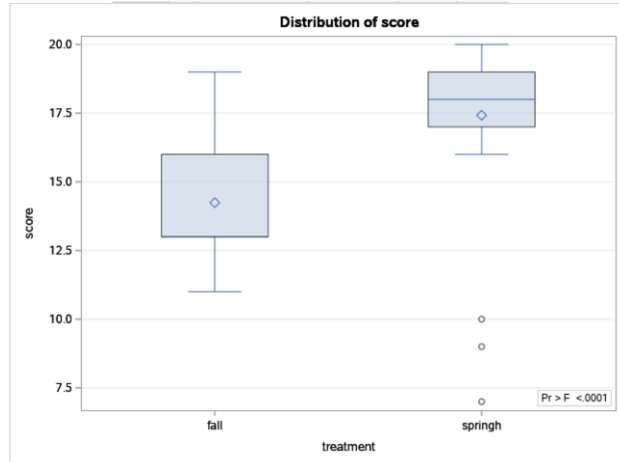


Figure 1: Treatment group vs control group post-test

6 Discussion

6.1 Overall gains

We found that while both groups exhibited gains in all 3 categories as seen in Table 1 (overall average scores went from 0.25 to 0.84 for the treatment group while the control group had a significantly smaller increase, from 0.23 to 0.33), the cognitive gains by the treatment group were also significantly larger than those of the control group in each of the three topics individually. Gains in unit testing were 55% (treatment) vs 9% (control); 57% (treatment) vs 8% (control) for integration testing; and 65%

(treatment) vs 9% (control) for continuous integration. We believe that these differences in cognitive gains between the treatment and control groups clearly show an effect of the PI methodology in student understanding and retention as evidenced by their pre-post test scores.

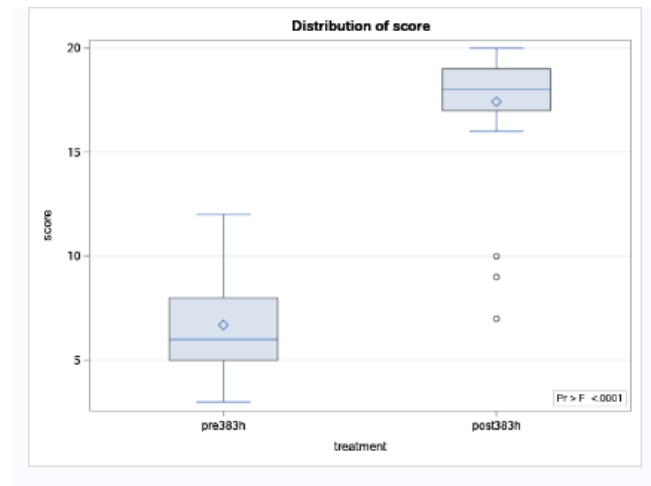


Figure 2: Treatment group - pre vs post-test

6.2 Prior academic exposure to topic

For unit testing, while the treatment group had larger gains, the control group exhibited marginally increased gains as compared to integration testing and continuous integration. We surmise that this could be due to the fact that both groups had been exposed to the topic in an earlier academic course. The treatment group had been exposed to unit testing through PI in the prior academic course. This could possibly explain their high pre-test scores on Q1 (64%) and Q3 (52%). This previous knowledge could have positively impacted their PI discussions during class, and subsequently, their post test scores for unit testing resulting in large gains (55%).

We found the largest gains in integration testing and continuous integration. One possible reason for this could be that in the topics of integration testing and continuous integration, students had little to no prior knowledge, with not having been exposed to those concepts either through lecture or PI in their academic preparation before this study.

Our survey contained questions linking coupling with testability (Q5), software design with unit testing (Q6), and software architecture with integration testing (Q7, Q8). The questions about integration testing and how databases can be tested without using actual databases (Q7, Q8) required an ability to weave together concepts regarding implementation variations of testing doubles with concepts from database modeling. All these questions lend themselves to lively discussions during PI, with those benefits possibly reflecting in the scores/gains.

6.3 Affect and PI

Our study found a significant difference on the impact of instructional method when considering student attitude regarding gender and usefulness in studying CS. Increased post-test scores in

the gender construct indicated a shift in student perceptions towards a belief of women being as capable and as competent in CS as men. We think that PI might have helped students see the big picture in SE, and thus impacted their views positively on how useful they consider CS to be in their careers. With the usefulness construct, these scores implied a shift in student attitudes towards computing skills being important in helping further their career goals.

We found no significant difference in the other constructs and we think this might be due to the fact that students in both the control group and treatment group were strongly motivated to learn computer science and SE, and that the choice of pedagogy did not necessarily increase or decrease their enjoyment of the study of these topics.

6.4 Comparison with previous PI studies in CS

We reinforce findings by Porter [34–36], Simon [37] and Lee [28] that PI had a positive impact on student affect. As for cognitive gains, previous studies have shown improvement in final exam performance by half a letter grade using PI in CS0 [37], learning gains of 41% in CS1 and 35% in CS1.5, and a study of a remedial CS1 course by Zingaro [40] reported learning gains of 29%. Porter et al. [36] found gains of up to 13% in the upper-level courses in their study, computer architecture and theory of computation. We found significant gains, as seen in Table 1. Noteworthy differences between the previous studies and ours include:

- 1) Several previous studies measured learning gains during each PI session, i.e., during each single class session [36, 37, 40], while our pre and post-tests were given at the beginning and end of the entire course.
- 2) Our measure of learning was not on individual PI questions, but on the test instrument we created. Previous studies used isomorphic questions [36, 40] while our study used an identical pre and post-test.
- 3) The metrics used to calculate learning gains were also different in these different studies [28, 34, 36] and it might not be possible to compare the gains demonstrated in earlier studies to those in ours. Normalized learning gain [28] was computed as the difference between the group vote and the preceding individual vote within each PI session, whereas our study utilized score gains based on the percentage of students who answered a question correctly.

6.5 Threats to validity

There are several uncontrollable and unknown factors that play into student performance in a class. Simon et al. [37] suggested in their study that students are “used” to standard instruction models (and how to succeed in them), and it might take more than one term to “train students” to be effective learners in such a radically different classroom model such as with PI. In the context of this study, the students in both groups were exposed to PI in their previous academic courses. However, the implementation details of the pedagogical technique might be different between those previous courses and the SE course that this study is conducted in. We intend to investigate this further in future iterations of our study.

Another threat to the validity of this study is the fact that we collected data from a single course, across two iterations, with a small number of students in each iteration (treatment $n=33$, control $n=34$). However, we intend to repeat this pilot study with other SE courses and with larger numbers of students in the future.

Finally, we used multiple choice responses for the control group rather than open-ended questions. It is possible that some of the distractors could have fooled students in a way that the open-ended questions did not. Students in both groups could have been exposed any of the topics studied in their prior industry internships. We had not collected data regarding internships in this iteration of our study. Further research is warranted to determine the influence of these factors, if any.

7 Conclusion

In this paper, we have presented findings from a quasi-experimental pilot study in SE, focused on the application of the PI pedagogy in teaching unit testing, integration testing and continuous integration. We saw significant score gains in the PI class in all three areas, and maximum gains in integration testing and continuous integration. The lecture class did not exhibit learning gains that were statistically significant. In this context, we believe that PI has a positive role to play in impacting student learning in these topics.

In response to RQ1: We found that PI does help undergraduate students in an SE class learn the concepts of unit testing, integration testing and continuous integration better than their counterparts in a purely lecture format class.

In response to RQ2: We found that there was an overall increase in enjoyment of students learning CS, with specific perception shifts being pronounced in the gender and usefulness constructs of the affective survey.

Some other areas for future work involve addressing questions such as: What, if any, is the role of industry internships in learning unit testing, integration testing and continuous integration? What role did peer discussion play in attaining these cognitive gains? We hope to address some of these questions in our future work and wish to emphasize the need for repeatability to verify our findings from this study.

REFERENCES

- [1] ACM Computing Curricula Task Force (Ed.). 2013. *Computer science curricula 2013: Curriculum guidelines for undergraduate degree programs in computer science*. ACM.
- [2] Adawi, T., Burden, H., Olsson, D., and Mattiasson, R. 2016. Characterizing software engineering students’ discussions during peer instruction: Opportunities for learning and implications for teaching. *Int. J. Eng. Educ.* 32, 2, 927–936.
- [3] Agarwal, R., Edwards, S.H., and Pérez-Quiriones, M. 2006. Designing an adaptive learning module to teach software testing. In *Proceedings of the 37th SIGCSE Technical Symposium on Computer Science Education*, 259–263.
- [4] Aniche, M., Hermans, F., and Deursen, A. 2019. Pragmatic software testing education. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education (SIGCSE '19)*, ACM, New York, NY, USA, 414–420.
- [5] Banerjee, M., Capozzoli, M., McSweeney, L., and Sinha, D. 1999. Beyond kappa: A review of interrater agreement measures. *Can. J. Stat.* 27, 1 (1999), 3–23.
- [6] Baptista, A. 2017. Report: Software failures cost \$1.1 trillion in 2016. *Service Virtualization Today*. Retrieved August 7, 2019 from <http://servicevirtualization.com/report-software-failures-cost-1-1-trillion-2016/>

- [7] Bertolino, A. 2007. Software testing research: Achievements, challenges, dreams. In *Future of Software Engineering (FOSE '07)*, IEEE, Minneapolis, MN, USA, 85–103.
- [8] Bloom, B. S. 1956. *Taxonomy of educational objectives, Handbook 1: Cognitive domain* (2nd edition Edition edition ed.). Addison-Wesley Longman Ltd, New York, NY, USA.
- [9] Buffardi, K. and Edwards, S.H. 2014. A formative study of influences on student testing behaviors. In *Proceedings of the 45th ACM Technical Symposium on Computer Science Education (SIGCSE '14)*, ACM, New York, NY, USA, 597–602.
- [10] Chen, Z., Zhang, J., and Luo, B. 2011. Teaching software testing methods based on diversity principles. In *2011 24th IEEE-CS Conference on Software Engineering Education and Training (CSEE T)*, Honolulu, HI, USA, 391–395.
- [11] Clarke, P.J., Pava, J., Davis, D., Hernandez, F., and King, T.M. 2012. Using wrest in se courses: An empirical study. In *Proceedings of the 43rd ACM Technical Symposium on Computer Science Education (SIGCSE '12)*, ACM, New York, NY, USA, 307–312.
- [12] Cutts, Q., Esper, S., and Simon, B. 2011. Computing as the 4th “R”: A general education approach to computing education. In *Proceedings of the Seventh International Workshop on Computing Education Research (ICER '11)*, ACM, New York, NY, USA, 133–138.
- [13] Drake, J.M. 2003. Drake, J., 2003. Teaching Software Testing: Lessons learned.DOI=(http://www.micsymposium.org/mics_2003/Drake.PDF).
- [14] Drost, E.A. 2011. Validity and reliability in social science research. *Educ. Res. Perspect.* 38, 1, 105–123.
- [15] Eddy, B.P., Wilde, N., Cooper, N.A., Mishra, B., Gamboa, V.S., Shah, K.S., Deleon, A.M. and Shields, N.A. 2017. A pilot study on introducing continuous integration and delivery into undergraduate software engineering courses. In *IEEE 30th Conference on Software Engineering Education and Training (CSEE&T)*, 47–56.
- [16] Edwards, S.H. 2004. Using software testing to move students from trial-and-error to reflection-in-action. In *Proceedings of the 35th SIGCSE Technical Symposium on Computer Science Education*, 26–30.
- [17] Esper, S. 2014. A discussion on adopting peer instruction in a course focused on risk management. *J. Comput. Sci. Coll.* 29, 4, 175–182.
- [18] Garousi, V. and Zhi, J. 2013. A survey of software testing practices in Canada. *J. Syst. Softw.* 86, 1354–1376.
- [19] Goldwasser, M.H. 2002. A gimmick to integrate software testing throughout the curriculum. In *Proceedings of the 33rd SIGCSE Technical Symposium on Computer Science Education*, 271–275.
- [20] Gopal, B. 2020. *Peer instruction questions for unit testing, integration testing and continuous integration*. <https://cse.unl.edu/~bgopal/PISWT.html>
- [21] Greising, L., Bartel, A. and Hagel, G. 2018. Introducing a deployment pipeline for continuous delivery in a software architecture course. In *Proceedings of the 3rd European Conference of Software Engineering Education*, 102–107.
- [22] Henderson, P.B. 2013. CS2013. *ACM Inroads* 4, 3 (2013), 48–49.
- [23] Hoegh, A. and Moskal, B. M. 2009. Examining science and engineering students’ attitudes toward computer science. In *2009 39th IEEE Frontiers in Education Conference*, 1–6.
- [24] Jones, E.L. 2001. Integrating testing into the curriculum — Arsenic in small doses. In *Proceedings of the Thirty-second SIGCSE Technical Symposium on Computer Science Education*, 337–341.
- [25] Kinne, J., Misner, E., Carter, A.S. and Tuttle, S.M. 2018. Evaluating the efficacy of clicker-based peer instruction across multiple courses at a single institution. *J. Comput. Sci. Coll.* 34, 1, 164–170.
- [26] Koretsky, M.D., Brooks, B.J., White, R.M. and Bowen, A. S. 2016. Querying the questions: Student responses and reasoning in an active learning class. *J. Eng. Educ.* 105, 2 (2016), 219–244.
- [27] Krathwohl, D. R. and Anderson, L.W. 2009. *A taxonomy for learning, teaching, and assessing: A revision of Bloom’s taxonomy of educational objectives*. Longman, New York, NY, USA.
- [28] Lee, C.B., Garcia, S. and Porter, L. 2013. Can peer instruction be effective in upper-division computer science courses? *Trans Comput Educ* 13, 3, 12:1–12:22.
- [29] Liao, S., Griswold, W. and Porter, L. 2017. Impact of class size on student evaluations for traditional and peer instruction classrooms. In *Proceedings of the ACM SIGCSE Technical Symposium on Computer Science Education*, ACM, 375–380.
- [30] Mazur, E. 1997. *Peer instruction: A user’s manual*. Prentice Hall, Upper Saddle River, NJ, USA.
- [31] Ng, S.P., Murnane, T., Reed, K., Grant, D., and Chen, T.Y. 2004. A preliminary survey on software testing practices in Australia. In *Australian Software Engineering Conference Proceedings*, IEEE, Melbourne, Vic., Australia, 116–125.
- [32] Orso, A. and Rothermel, G. 2014. Software testing: A research travelogue (2000–2014). In *Proceedings of the on Future of Software Engineering*, ACM, 117–132.
- [33] Pargas, R.P. and Shah, D.M. 2006. Things are clicking in computer science courses. In *Proceedings of the 37th SIGCSE Technical Symposium on Computer Science Education*, ACM, 474–478.
- [34] Leo Porter, Cynthia Bailey Lee, and Beth Simon. 2013. Halving fail rates using peer instruction: A study of four computer science courses. In *Proceeding of the 44th ACM Technical Symposium on Computer Science Education*, ACM, 177–182.
- [35] Porter, L., Lee, C.B., Simon, B., Cutts, Q. and Zingaro, D. 2011. Experience report: A multi-classroom report on the value of peer instruction. In *Proceedings of the 16th Annual Joint Conference on Innovation and Technology in Computer Science Education*, ACM, 138–142.
- [36] Porter, L., Lee, C.B., Simon, B., and Zingaro, D. 2011. Peer instruction: Do students really learn from peer discussion in computing? In *Proceedings of the Seventh International Workshop on Computing Education Research*, ACM, 45–52.
- [37] Simon, B., Parris, J., and Spacco, J. 2013. How we teach impacts student learning: Peer instruction vs. lecture in cs0. In *Proceeding of the 44th ACM Technical Symposium on Computer Science Education*, ACM, 41–46.
- [38] Tan, N., Shoemaker, G., Gedi, A., Mache, J., and Weiss, R. 2017. Applying a framework for creating and analyzing cybersecurity questions for peer instruction. *J. Comput. Sci. Coll.* 33, 1, 102–108.
- [39] Wong, W.E., Bertolino, A., Debroy, V., Mathur, A., Offutt, J. and Vouk, M. 2011. Teaching software testing: Experiences, lessons learned and the path forward. In *24th IEEE-CS Conference on Software Engineering Education and Training*, 530–534.
- [40] Zingaro, D. 2010. Experience report: Peer instruction in remedial computer science. *Association for the Advancement of Computing in Education (AACE)*, 5030–5035. Retrieved August 6, 2019 from <https://www.learntechlib.org/primary/p/36184/USA>, 356–361.